Optimizing Matrix Transpose in Transformer Self-Attention

CUDA Performance Analysis Report

September 8, 2026

1 Introduction

This report presents the design, implementation, and performance evaluation of two CUDA transpose kernels used within a simplified Transformer self-attention pipeline. The pipeline computes the Key matrix transpose (K^T), the query–key product (QK^T), the softmax of the resulting attention scores, and the final attention output. The goal of this assignment is to **replace the naive (baseline) transpose kernel and the naive multiplication kernel with an optimized shared-memory tiled transpose kernel and optimised multiplication kernel**, and quantify the performance impact of the optimization on both the isolated transpose kernel and the end-to-end pipeline.

2 Results

2.1 Summary Execution Statistics ($N = 4096$)

Table 1: End-to-end benchmark comparison between the original and optimized pipelines(from Google Colab experimentation)

Statistic	**Original**	**Optimized**
Transpose execution time (ms)	1	0.6858
End-to-end execution time (ms)	534	362
HtoD transfer time (ms)	51.1199	43.5168
HtoD data transferred (MB)	192.0000	192.0000
End-to-end speedup	1.00×	1.47×

2.2 nsys Kernel Summary

Table 2: CUDA GPU kernel summary – Original pipeline (`cuda_gpu_kern_sum`).

Kernel	**Time (%)**	**Total Time (ns)**	**Instances**
matMulKernel	99.8	1,000,087,846	2
transposeKernel	0.1	1,079,770	1
softmaxKernel	0.1	753,691	1

Table 3: CUDA GPU kernel summary – Optimized pipeline (`cuda_gpu_kern_sum`).

Kernel	Time (%)	Total Time (ns)	Instances
matMulKernel	99.2	126,351,504	2
softmaxKernel	0.6	752,124	1
transposeKernel_Optimized	0.2	251,422	1

2.3 nsys Memory-Operation Timing

Table 4: CUDA GPU memory-operation timing (`cuda_gpu_mem_time_sum`).

Version	Operation	Time (%)	Total Time (ns)	Count
Original	memcpy DtoH	81.7	244,370,676	4
Original	memcpy HtoD	18.3	54,917,593	3
Optimized	memcpy DtoH	83.0	250,515,438	4
Optimized	memcpy HtoD	17.0	51,433,486	3

2.4 nsys Memory-Transfer Size

Table 5: CUDA GPU memory-transfer sizes (`cuda_gpu_mem_size_sum`), identical for both versions since data volume is unaffected by the transpose algorithm.

Operation	Total (MB)	Count	Avg (MB)
memcpy DtoH	268.435	4	67.109
memcpy HtoD	201.327	3	67.109

3 Conclusion

The shared-memory tiled transpose kernel achieved a 1.4–4.3× improvement over the naive baseline at the kernel level, depending on whether host-side or device-side (nsys) timing is considered, by converting an uncoalesced global-memory write pattern into fully coalesced reads and writes via a padded on-chip tile. This highlights that, in attention-style pipelines dominated by large GEMM operations, transpose optimization primarily benefits workloads where the transpose is repeated frequently, performed on very large matrices, or where the matrix multiplication itself is already well optimized (e.g., using tensor cores or cuBLAS), at which point the transpose's relative contribution to total time grows and further optimization becomes more impactful.